

```

sql> /*
-- =====
-- ■ DDL (Data Definition Language)
-- General Syntax (SQL Comments Only)
-- =====

-- ♦ CREATE DATABASE
CREATE DATABASE database_name;

-- ♦ USE DATABASE
USE database_name;

-- ♦ CREATE TABLE
CREATE TABLE table_name (
    column_name1 datatype [constraint],
    column_name2 datatype [constraint],
    ...
);

-- ♦ ALTER TABLE - ADD COLUMN
ALTER TABLE table_name
ADD COLUMN column_name datatype;

-- ♦ ALTER TABLE - MODIFY COLUMN
ALTER TABLE table_name
MODIFY COLUMN column_name new_datatype;

-- ♦ ALTER TABLE - RENAME TABLE
RENAME TABLE old_table_name TO new_table_name;

-- ♦ TRUNCATE TABLE (Deletes all rows, keeps structure)
TRUNCATE TABLE table_name;

-- ♦ DROP TABLE (Deletes table and data)
DROP TABLE table_name;

-- Safer version (no error if table doesn't exist)
DROP TABLE IF EXISTS table_name;

*/

```

```

•CREATE DATABASE sql_zero_to_hero;

```

```

sql> •USE sql_zero_to_hero;
OK, 0 records retrieved in 0.976ms

```

```

sql> •CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    city VARCHAR(50),
    email VARCHAR(100)
);
OK, 0 records retrieved in 86.278ms

```

```

sql> •INSERT INTO students (student_id, name, age, city, email)
VALUES
(1, 'Suhas Shende', 23, 'Pune', 'suhas@example.com'),
(2, 'Rutuja Wagh', 22, 'Nagpur', 'rutuja@example.com'),
(3, 'Kewal Patil', 24, 'Mumbai', 'kewal@example.com'),
(4, 'Chirag Jain', 23, 'Nanded', 'chirag@example.com'),
(5, 'Sonal Mehta', 21, 'Aurangabad', 'sonal@example.com');
OK, 5 rows affected in 29.633ms

```

```

sql> •SELECT * FROM students;

```

OK, 5 records retrieved in 1.475ms

```

sql> -- Add a new column
•ALTER TABLE students ADD COLUMN phone_number VARCHAR(15);
OK, 0 records retrieved in 71.623ms

```

```

sql> •SELECT * FROM students;

```

student_id	name	age	city	email	phone_number
1	Suhas Shende	23	Pune	suhas@example.com	NULL
2	Rutuja Wagh	22	Nagpur	rutuja@example.com	NULL
3	Kewal Patil	24	Mumbai	kewal@example.com	NULL
4	Chirag Jain	23	Nanded	chirag@example.com	NULL
5	Sonal Mehta	21	Aurangabad	sonal@example.com	NULL

OK, 5 records retrieved in 1.788ms

```

sql> -- Modify column data type
•ALTER TABLE students MODIFY COLUMN age TINYINT;
OK, 5 rows affected in 178.278ms

```

```

sql> -- describe the data type
•desc students;

```

Field	Type	Null	Key	Default	Extra
student_id	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
age	tinyint	YES		NULL	
city	varchar(50)	YES		NULL	
email	varchar(100)	YES		NULL	
phone_number	varchar(15)	YES		NULL	

OK, 6 records retrieved in 3.271ms

```

sql> -- Rename the table
•RENAME TABLE students TO college_students;
OK, 0 records retrieved in 63.308ms

```

```

sql> -- Delete all data (structure remains)
•TRUNCATE TABLE college_students;
OK, 0 records retrieved in 144.927ms

```

```

sql> -- Delete entire table (structure + data gone)
•DROP TABLE IF EXISTS college_students;
OK, 0 records retrieved in 52.497ms

```

```

sql> /*
-- 📌 TOPIC: SQL CONSTRAINTS (with Real-world Tables)
-----
-- SQL Constraints are rules applied on columns to control what type of data can be stored in a table.
-- They ensure accuracy, consistency, and integrity of data in the database.

-- ✅ Common SQL Constraints:

-- 1. PRIMARY KEY:
--    - Uniquely identifies each row in a table.
--    - Must be UNIQUE and NOT NULL.

-- 2. FOREIGN KEY:
--    - Ensures referential integrity by linking to a primary key in another table.

-- 3. NOT NULL:
--    - Prevents null (empty) values in a column.

-- 4. UNIQUE:
--    - Ensures all values in a column are distinct.

-- 5. CHECK:
--    - Validates that a value meets a specific condition.

-- 6. DEFAULT:
--    - Assigns a default value if none is provided during INSERT.

-- 7. AUTO-INCREMENT:
--    - Automatically increases the value of a numeric column for each new row (often used with PRIMARY KEY).

-- 🎯 Why Use Constraints?
--    - To avoid duplicate, invalid, or incomplete data.
--    - To enforce business rules at the database level.
--    - To protect relationships between tables (e.g., products must belong to valid categories).

-- 📌 Example below uses: categories (parent) and products (child) tables with realistic constraints.

-- 🔴 CHECK CONSTRAINT in SQL
-----
-- The CHECK constraint allows you to define rules (conditions)
-- that must be TRUE for the data to be inserted or updated in a column.

-- ✅ Use Cases:
--    - Ensure valid numeric ranges (e.g., price > 0).
--    - Prevent negative values (e.g., quantity >= 0).
--    - Restrict allowed values (e.g., status IN ('Paid', 'Pending')).

-- 🔴 Syntax (inline):
--    column_name datatype CHECK (condition)

-- 🔴 Syntax (table-level):
--    CHECK (condition on one or more columns)

-----
-- ✅ Examples:

-- 1. Ensure age is at least 18:
--    age INT CHECK (age >= 18)

-- 2. Ensure quantity is not negative:
--    quantity INT CHECK (quantity >= 0)

-- 3. Allow only 3 food payment status values:
--    payment_status VARCHAR(20) CHECK (payment_status IN ('Paid', 'Pending', 'Failed'))

-- ⚠️ If a row violates the CHECK condition, it will not be inserted or updated.

-----

*/

OK, 0 records retrieved in 0.559ms

sql> use sql_zero_to_hero;

OK, 0 records retrieved in 0.798ms

sql> -- 📌 1. Create categories Table (WITH PRIMARY KEY)
+CREATE TABLE categories (
+  category_id INT PRIMARY KEY AUTO_INCREMENT,
+  category_name VARCHAR(100) NOT NULL UNIQUE
+);

OK, 0 records retrieved in 90.896ms

sql> -- Insert categories
+INSERT INTO categories (category_name)
+VALUES ('Electronics'), ('Clothing'), ('Furniture');

OK, 3 rows affected in 23.69ms

sql> desc categories;

+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| category_id | int | NO | PRI | NULL | auto_increment |
| category_name | varchar(100) | NO | UNI | NULL | |
+-----+-----+-----+-----+-----+-----+

OK, 2 records retrieved in 2.72ms

View | Pages | Info | ...

sql> -- 📌 2. Create products Table (with Constraints)
+CREATE TABLE products (
+  product_id INT PRIMARY KEY AUTO_INCREMENT,
+  product_name VARCHAR(100) NOT NULL,
+  price DECIMAL(10,2) CHECK (price > 0), -- Must be positive
+  quantity INT DEFAULT 0 CHECK (quantity >= 0), -- Foreign key
+  category_id INT, -- Foreign key
+  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
+  FOREIGN KEY (category_id) REFERENCES categories(category_id)
+);

OK, 0 records retrieved in 114.853ms

sql> -- 📌 3. Insert Valid Products
+INSERT INTO products (product_name, price, quantity, category_id)
+VALUES
+  ('iPhone 15', 7999.99, 10, 1),
+  ('Cotton Shirt', 1299.00, 50, 2),
+  ('Office Desk', 4500.00, 5, 3);

OK, 3 rows affected in 21.43ms

sql> desc products;

+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| product_id | int | NO | PRI | NULL | auto_increment |
| product_name | varchar(100) | NO | | NULL | |
| price | decimal(10,2) | YES | | NULL | |
| quantity | int | YES | | 0 | |
| category_id | int | YES | MUL | NULL | |
| created_at | datetime | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+

OK, 6 records retrieved in 2.584ms

View | Pages | Info | ...

sql> SELECT * FROM categories;

+-----+-----+
| category_id | category_name |
+-----+-----+
| 2 | Clothing |
| 1 | Electronics |
| 3 | Furniture |
+-----+-----+

OK, 3 records retrieved in 3.127ms

View | Pages | Info | ...

sql> SELECT * FROM products;

+-----+-----+-----+-----+-----+-----+
| product_id | product_name | price | quantity | category_id | created_at |
+-----+-----+-----+-----+-----+-----+
| 1 | iPhone 15 | 7999.99 | 10 | 1 | 07/08/2025 |
| 2 | Cotton Shirt | 1299.00 | 50 | 2 | 07/08/2025 |
| 3 | Office Desk | 4500.00 | 5 | 3 | 07/08/2025 |
+-----+-----+-----+-----+-----+-----+

OK, 3 records retrieved in 1.313ms

View | Pages | Info | ...

```

```
sql> /
-- MySQL: DATA TYPES SUMMARY
-----
MySQL supports multiple data types grouped into 4 main categories:

-- 1. NUMERIC DATA TYPES

-- INT      + Integer values (e.g., 1, 1000)
-- TINYINT  + Small integers (-128 to 127 or 0 to 255 unsigned)
-- FLOAT(M,D) + Approximate decimal (e.g., 172.55)
-- DOUBLE(M,D) + Larger approximate decimal values
-- DECIMAL(M,D) + Exact decimal (e.g., 124578.25), used for currency, finance

-- 2. STRING / TEXT TYPES

-- CHAR(n)   + Fixed-length string (always n characters, padded with spaces)
-- VARCHAR(n) + Variable-length string (up to n characters)
-- TEXT      + Large text up to ~84KB (can't have default value)

-- 3. DATE / TIME TYPES

-- DATE      + Only date (YYYY-MM-DD)
-- TIME      + Only time (HH:MM:SS)
-- DATETIME  + Date + Time (YYYY-MM-DD HH:MM:SS)
-- TIMESTAMP + Auto-updated datetime, often used for last modified
-- YEAR      + 4-digit year (e.g., 2025)

-- 4. BOOLEAN

-- BOOLEAN  + Alias for TINYINT(1), stores TRUE as 1 and FALSE as 0

-----
-- Best Practices:

-- Use CHAR(n) for codes (e.g., gender = 'M'/'F', country codes)
-- Use VARCHAR(n) for names, emails, etc.
-- Use DECIMAL for financial amounts
-- Use FLOAT/DOUBLE for scientific or approximate numbers
-- Use DATE or DATETIME for time-based tracking
-- BOOLEAN is best for status flags like is_active, is_deleted, etc.

-----
# MySQL Data Types Summary

## Numeric Types

| Type      | Description                                     | Example      |
|-----|-----|-----|
| INT       | Whole number (4 bytes)                         | 100, 2025   |
| TINYINT   | Small int (-128 to 127)                        | 1, 128      |
| FLOAT(5,2) | Approx decimal (5 digits, 2 after .)          | 172.55      |
| DOUBLE    | Large decimal (approximate)                    | 123456.78   |
| DECIMAL(10,2) | Precise decimal (10 digits, 2 decimal)    | 124578.25   |

## String / Text Types

| Type      | Description                                     | Example      |
|-----|-----|-----|
| CHAR(3)   | Fixed-length string (padded)                  | 'ABC', 'X '  |
| VARCHAR(50) | Variable-length string                        | 'Suhas', 'Email' |
| TEXT      | Long text up to ~84KB                         | Bio, comments |

## Date and Time Types

| Type      | Description                                     | Example      |
|-----|-----|-----|
| DATE      | Only date                                       | '2025-07-08' |
| DATETIME  | Date and time                                  | '2025-07-08 10:00:00' |
| TIMESTAMP | Auto date-time (updated)                      | '2025-07-08 10:00:00' |
| TIME      | Only time                                       | '14:30:00'   |
| YEAR      | Only year                                       | '2025'       |

## Boolean

| Type      | Description                                     | Stored As |
|-----|-----|-----|
| BOOLEAN   | TRUE/FALSE (alias TINYINT(1))                 | 1 / 0       |

-----

*/
OK, 0 records retrieved in 0.693ms

sql> use sql_zero_to_hero;
OK, 0 records retrieved in 0.804ms

sql> DROP TABLE IF EXISTS datatype_demo;
OK, 0 records retrieved in 109.881ms

sql> CREATE TABLE datatype_demo (
  id INT PRIMARY KEY AUTO_INCREMENT,
  age TINYINT,
  name VARCHAR(50),
  code CHAR(3),
  description TEXT,
  dob DATE,
  created_at DATETIME,
  is_active BOOLEAN,
  height_cm FLOAT(5,2),
  salary DOUBLE(10,2),
  bank_balance DECIMAL(12,2) -- DECIMAL: Exact numeric (good for money)
);
OK, 0 records retrieved in 87.19ms

sql> INSERT INTO datatype_demo
(age, name, code, description, dob, created_at, is_active, height_cm, salary, bank_balance)
VALUES
(25, 'Suhas', 'ABC', 'This is a sample text', '1999-06-15', '2024-01-01 10:30:00', TRUE, 172.55, 55000.75, 124578.25);

OK, 1 row affected in 27.202ms

sql> select * from datatype_demo;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | age | name | code | description | dob | created_at | is_active | height_cm | salary | bank_balance |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | 25  | Suhas | ABC  | This is a sample text | 06/15/1999 | 01/01/2024 | true | 172.55 | 55000.75 | 124578.25 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

OK, 1 record retrieved in 1.532ms

-- sql> select * from datatype_demo;
```

```

-- SQL commands modify table data without changing the table structure.
-- Table structure
-- > CREATE || add new data
-- > UPDATE || modify existing data
-- > DELETE || remove data
-- > INSERT INTO SELECT || copy data from one table to another
-- > INSERT INTO || insert values into table

-- > INSERT Syntax
-- INSERT INTO table_name (column1, column2, ...)
-- VALUES (value1, value2, ...);

-- > INSERT Syntax
-- INSERT INTO table_name (column1, column2, ...)
-- VALUES
-- (value1, value2, ...),
-- (value1, value2, ...);

-- > UPDATE Syntax
-- UPDATE table_name
-- SET column1 = value1, column2 = value2, ...
-- WHERE condition;

-- > UPDATE Syntax
-- UPDATE table_name
-- SET column1 = value1, column2 = value2, ...
-- WHERE condition;

-- > INSERT INTO SELECT Syntax
-- Copy data from one table to another
-- INSERT INTO target_table (column1, column2, ...)
-- SELECT column1, column2, ...
-- FROM source_table;
-- WHERE condition;

-- > UPDATE with JOIN Syntax
-- Update table using data from another table
-- UPDATE table1 AS t1
-- SET column1 = t2.column2
-- FROM table2 AS t2
-- WHERE t1.column1 = t2.column2;
-- WHERE condition;

-- > Backup Example
-- Increase price by 10% for products where stock
-- UPDATE products
-- SET price = price * 1.10;
-- WHERE price < 1000;
-- /

OK, 0 records retrieved in 0.141ms

-- SQL query, pers. to here;

OK, 0 records retrieved in 1.203ms

-- > Create categories table
-- CREATE TABLE IF NOT EXISTS categories_04 (
-- category_id INT PRIMARY KEY AUTO_INCREMENT,
-- category_name VARCHAR(100) NOT NULL,
-- );

OK, 0 records retrieved in 100.67ms

-- > Create products table
-- CREATE TABLE IF NOT EXISTS products_04 (
-- product_id INT PRIMARY KEY AUTO_INCREMENT,
-- product_name VARCHAR(100) NOT NULL,
-- price DECIMAL(10,2),
-- stock INT,
-- category_id INT,
-- FOREIGN KEY (category_id) REFERENCES categories(category_id)
-- );

OK, 0 records retrieved in 123.33ms

-- > 1. INSERT - add records
-- INSERT INTO categories_04 (category_name)
-- VALUES ('Electronics'), ('Books'), ('Clothing');

OK, 3 rows affected in 10.40ms

-- > Insert into products
-- INSERT INTO products_04 (product_name, price, stock, category_id)
-- VALUES
-- ('iPhone 15', 7000.00, 15, 1),
-- ('Wireless Mouse', 499.00, 100, 1),
-- ('Smartwatch', 1000.00, 50, 2),
-- ('Novel - Atomic Habits', 450.00, 30, 3);

OK, 4 rows affected in 25.10ms

-- > 2. UPDATE - modify data
-- Modify stock of a product
-- UPDATE products_04
-- SET stock = stock - 5;
-- WHERE product_name = 'Smartwatch';

OK, 1 row affected in 10.72ms

-- > Change price for all 'Smartwatch'
-- UPDATE products_04
-- SET price = price * 1.10;
-- WHERE product_id = 1;

OK, 1 row affected in 10.72ms

-- > 3. DELETE - remove records
-- Delete a product with low price
-- DELETE FROM products
-- WHERE price < 500;

OK, 0 records retrieved in 3.10ms

-- > SELECT - view categories_04;
-- category_id | category_name
-- 1 | Electronics
-- 2 | Books
-- 3 | Clothing

OK, 3 records retrieved in 0.00ms

-- > SELECT - view products_04;
-- product_id | product_name | price | stock | category_id
-- 1 | Electronics - Electronics - iPhone 15 | 7000.00 | 15 | 1
-- 2 | Electronics - Electronics - Wireless Mouse | 499.00 | 100 | 1
-- 3 | Smartwatch | 1000.00 | 45 | 2
-- 4 | Novel - Atomic Habits | 450.00 | 30 | 3

OK, 4 records retrieved in 0.00ms

-- > 4. INSERT INTO SELECT - Copy data from one table to another
-- CREATE TABLE IF NOT EXISTS products_backup_04;
-- SELECT * FROM products_04 WHERE stock < 100;
-- /

OK, 0 records retrieved in 0.01ms
OK, 0 records retrieved in 0.03ms

-- > Backup products where stock < 1000 into backup
-- INSERT INTO product_backup
-- SELECT * FROM products_04
-- WHERE price < 1000;

OK, 0 rows affected in 10.66ms

-- > SELECT - view product_backup;
-- product_id | product_name | price | stock | category_id
-- 1 | iPhone 15 | 7000.00 | 15 | 1
-- 3 | Smartwatch | 1000.00 | 45 | 2

OK, 2 records retrieved in 0.00ms

-- > 5. UPDATE with JOIN - Update using related table
-- No all Electronics product names to price with category
-- UPDATE products_04 p
-- JOIN categories_04 c ON p.category_id = c.category_id
-- SET p.product_name = CONCAT(c.category_name, '-', p.product_name);
-- WHERE c.category_name = 'Electronics';

OK, 2 rows affected in 42.03ms

-- > SELECT - view products_04;
-- product_id | product_name | price | stock | category_id
-- 1 | Electronics - Electronics - iPhone 15 | 7000.00 | 15 | 1
-- 2 | Electronics - Electronics - Wireless Mouse | 499.00 | 100 | 1
-- 3 | Smartwatch | 1000.00 | 45 | 2
-- 4 | Novel - Atomic Habits | 450.00 | 30 | 3

```



```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

-- 01. Functions (mysql - database)
-- Built-in MySQL Functions with Examples
=====
Function      Desc      Example
-----
COUNT()     Count rows      SELECT COUNT(*)
AVG()         Average value   SELECT AVG(amount)
MAX()         Max value       SELECT MAX(amount)
MIN()         Min value       SELECT MIN(amount)
SUM()         Total           SELECT SUM(amount)
CONCAT()      Merge strings   SELECT CONCAT(name, city)
UPPER()       Uppercase       SELECT UPPER(name)
LOWER()       Lowercase       SELECT LOWER(name)
DATE_FORMAT() Format date      SELECT DATE_FORMAT(date, '%d-%b-%Y')
GROUP_CONCAT() Merge group values
=====

--
OK, 8 records retrieved in 0.0066s

sql> -- Step 1: Create sample table
-- CREATE TABLE orders_07 (
  order_id INT PRIMARY KEY,
  customer_name VARCHAR(50),
  amount DECIMAL(10,2),
  order_date DATE,
  city VARCHAR(10)
);

OK, 8 records retrieved in 0.0344s

sql> -- Step 2: Insert sample data
-- INSERT INTO orders_07 (order_id, customer_name, amount, order_date, city)
VALUES
(1, 'Suhas', 1980.50, '2024-12-01', 'Pune'),
(2, 'Rutuja', 2300.00, '2024-12-03', 'Mangur'),
(3, 'Kawal', 880.75, '2024-12-04', 'Mumbai'),
(4, 'Chirag', 1500.50, '2024-12-05', 'Nanded'),
(5, 'Sonali', 4200.25, '2024-12-06', 'Pune'),
(6, 'Suhas', 1300.00, '2024-12-07', 'Pune');

OK, 6 rows affected in 0.1029s

sql> -- Step 3: View all data
-- SELECT * FROM orders_07;

order_id  customer_name  amount  order_date  city
-----
1         Suhas         1980.50  12/01/2024  Pune
2         Rutuja        2300.00  12/03/2024  Mangur
3         Kawal         880.75   12/04/2024  Mumbai
4         Chirag        1500.50  12/05/2024  Nanded
5         Sonali        4200.25  12/06/2024  Pune
6         Suhas         1300.00  12/07/2024  Pune

OK, 6 records retrieved in 0.0410s

sql> -- COUNT()
-- Returns number of rows
-- Syntax: COUNT(column_name or *)
-- SELECT COUNT(*) AS total_orders FROM orders_07;

total_orders
-----
6

OK, 1 record retrieved in 0.0790s

sql> -- AVG()
-- Returns average of numeric column
-- Syntax: AVG(column_name)
-- SELECT AVG(amount) AS average_amount FROM orders_07;

average_amount
-----
1993.066667

OK, 1 record retrieved in 0.0720s

sql> -- MAX()
-- Returns highest value in column
-- Syntax: MAX(column_name)
-- SELECT MAX(amount) AS max_amount FROM orders_07;

max_amount
-----
4200.25

OK, 1 record retrieved in 0.0380s

sql> -- MIN()
-- Returns lowest value
-- Syntax: MIN(column_name)
-- SELECT MIN(amount) AS min_amount FROM orders_07;

min_amount
-----
880.75

OK, 1 record retrieved in 0.0320s

sql> -- SUM()
-- Returns total sum of column
-- Syntax: SUM(column_name)
-- SELECT SUM(amount) AS total_sales FROM orders_07;

total_sales
-----
13002.00

OK, 1 record retrieved in 0.0400s

sql> -- CONCAT()
-- Combines values from multiple columns/strings
-- Syntax: CONCAT(col1, col2, ...)
-- SELECT CONCAT(customer_name, ' - ', city) AS full_info FROM orders_07;

full_info
-----
Suhas - Pune
Rutuja - Mangur
Kawal - Mumbai
Chirag - Nanded
Sonali - Pune
Suhas - Pune

OK, 6 records retrieved in 0.0200s

sql> -- UPPER()
-- Converts text to uppercase
-- Syntax: UPPER(column_name)
-- SELECT UPPER(customer_name) AS name_upper FROM orders_07;

name_upper
-----
SUHAS
RUTUJA
KEWAL
CHIRAG
SONALI
SUHAS

OK, 6 records retrieved in 0.0280s

sql> -- LOWER()
-- Converts text to lowercase
-- Syntax: LOWER(column_name)
-- SELECT LOWER(city) AS city_lower FROM orders_07;

city_lower
-----
pune
mangur
mumbai
nanded
pune
pune

OK, 6 records retrieved in 0.0280s

sql> -- DATE_FORMAT()
-- Formats date into given format
-- Syntax: DATE_FORMAT(date, column 'format')
-- SELECT DATE_FORMAT(order_date, '%d-%b-%Y') AS formatted_date FROM orders_07;

formatted_date
-----
01-Dec-2024
03-Dec-2024
04-Dec-2024
05-Dec-2024
06-Dec-2024
07-Dec-2024

OK, 6 records retrieved in 0.0300s

sql> -- GROUP_CONCAT()
-- Combines multiple row values into a single string
-- Syntax: GROUP_CONCAT(column_name SEPARATOR ' ')
-- SELECT city, GROUP_CONCAT(customer_name) AS customers
FROM orders_07
GROUP BY city;

city      customers
-----
Mumbai1   Kawal
Mangur    Rutuja
Nanded    Chirag
Pune      Suhas,Sonali,Suhas

```

```

sql> */
-- *****
-- * GROUP BY having MySQL notebook
-- GROUP BY, HAVING, ROLLUP + All Aggregate Functions
-- *****

Aggregate Functions By
| Function | Purpose | Example |
|-----|-----|-----|
| COUNT() | Number of rows | COUNT(*) |
| SUM() | Total sum | SUM(amount) |
| AVG() | Average value | AVG(amount) |
| MIN() | Minimum value | MIN(amount) |
| MAX() | Maximum value | MAX(amount) |

*/

OK, 0 records retrieved in 1.889ms

sql> USE sql_zero_to_hero;
OK, 0 records retrieved in 0.733ms

sql>
-- Step 1: Create table
-- CREATE TABLE sales_08 (
--   sale_id INT PRIMARY KEY,
--   employee_name VARCHAR(50),
--   region VARCHAR(50),
--   amount DECIMAL(10,2),
--   sale_date DATE
-- );
OK, 0 records retrieved in 90.691ms

sql> -- Step 2: Insert sample data
-- INSERT INTO sales_08 (sale_id, employee_name, region, amount, sale_date)
-- VALUES
-- (1, 'Suhas', 'West', 1200.00, '2024-12-01'),
-- (2, 'Rutuja', 'East', 800.00, '2024-12-01'),
-- (3, 'Kewal', 'West', 950.00, '2024-12-02'),
-- (4, 'Chirag', 'North', 1500.00, '2024-12-02'),
-- (5, 'Sonal', 'East', 2000.00, '2024-12-03'),
-- (6, 'Suhas', 'West', 500.00, '2024-12-03'),
-- (7, 'Kewal', 'North', 1800.00, '2024-12-03'),
-- (8, 'Rutuja', 'East', 1300.00, '2024-12-04');
OK, 8 rows affected in 24.185ms

sql>
-- Step 3: View all records
-- SELECT * FROM sales_08;

+-----+-----+-----+-----+-----+
| sale_id | employee_name | region | amount | sale_date |
+-----+-----+-----+-----+-----+
| 1 | Suhas | West | 1200.00 | 2024-12-01 |
| 2 | Rutuja | East | 800.00 | 2024-12-01 |
| 3 | Kewal | West | 950.00 | 2024-12-02 |
| 4 | Chirag | North | 1500.00 | 2024-12-02 |
| 5 | Sonal | East | 2000.00 | 2024-12-03 |
| 6 | Suhas | West | 500.00 | 2024-12-03 |
| 7 | Kewal | North | 1800.00 | 2024-12-03 |
| 8 | Rutuja | East | 1300.00 | 2024-12-04 |
+-----+-----+-----+-----+-----+

OK, 8 records retrieved in 1.388ms

sql>
-- *****
-- * GROUP BY region with all aggregate functions
-- SELECT
--   region,
--   COUNT(*) AS total_orders,
--   SUM(amount) AS total_sales,
--   AVG(amount) AS average_sales,
--   MIN(amount) AS min_sale,
--   MAX(amount) AS max_sale
-- FROM sales_08
-- GROUP BY region;

+-----+-----+-----+-----+-----+
| region | total_orders | total_sales | average_sales | min_sale | max_sale |
+-----+-----+-----+-----+-----+
| West | 3 | 2650.00 | 883.333333 | 500.00 | 1200.00 |
| East | 3 | 4800.00 | 1333.333333 | 800.00 | 2000.00 |
| North | 2 | 3300.00 | 1650.000000 | 1500.00 | 1800.00 |
+-----+-----+-----+-----+-----+

OK, 3 records retrieved in 2.545ms

sql>
-- *****
-- * GROUP BY employee with HAVING (filtering aggregated results)
-- SELECT
--   employee_name,
--   COUNT(*) AS total_orders,
--   SUM(amount) AS total_sales
-- FROM sales_08
-- GROUP BY employee_name
-- HAVING total_sales > 2000;

+-----+-----+-----+
| employee_name | orders | total_sales |
+-----+-----+-----+
| Kewal | 2 | 2750.00 |
+-----+-----+-----+

OK, 1 record retrieved in 3.297ms

sql>
-- *****
-- * GROUP BY with ROLLUP (adds total row)
-- SELECT
--   region,
--   SUM(amount) AS total_sales
-- FROM sales_08
-- GROUP BY region WITH ROLLUP;

+-----+-----+
| region | total_sales |
+-----+-----+
| East | 4800.00 |
| North | 3300.00 |
| West | 2650.00 |
| ROW | 9950.00 |
+-----+-----+

OK, 4 records retrieved in 2.182ms

sql>
-- *****
-- * GROUP BY formatted month using DATE_FORMAT()
-- SELECT
--   DATE_FORMAT(sale_date, '%M') AS sale_month,
--   COUNT(*) AS total_orders,
--   SUM(amount) AS monthly_sales
-- FROM sales_08
-- GROUP BY sale_month;

+-----+-----+-----+
| sale_month | orders | monthly_sales |
+-----+-----+-----+
| December | 8 | 9950.00 |
+-----+-----+-----+

OK, 1 record retrieved in 3.957ms

sql>
-- *****
-- * GROUP BY derived column using CASE for amount ranges
-- SELECT
--   CASE
--     WHEN amount < 1000 THEN 'Low'
--     WHEN amount BETWEEN 1000 AND 1500 THEN 'Medium'
--     ELSE 'High'
--   END AS sale_level,
--   COUNT(*) AS sales_count,
--   SUM(amount) AS total_amount,
--   MAX(amount) AS max_amt
-- FROM sales_08
-- GROUP BY sale_level;

+-----+-----+-----+-----+
| sale_level | sales_count | total_amount | max_amt |
+-----+-----+-----+-----+
| Medium | 3 | 3900.00 | 1500.00 |
| Low | 3 | 2250.00 | 950.00 |
| High | 2 | 3300.00 | 2000.00 |
+-----+-----+-----+-----+

OK, 3 records retrieved in 2.81ms

sql>
-- * Grouping sales by employee_name and using HAVING to filter those
-- whose total sales are greater than 2000

-- SELECT
--   employee_name,
--   COUNT(*) AS total_orders,
--   SUM(amount) AS total_sales,
--   AVG(amount) AS avg_order_amount
-- FROM sales_08
-- GROUP BY employee_name
-- HAVING total_sales > 2000;

+-----+-----+-----+-----+
| employee_name | total_orders | total_sales | avg_order_amount |
+-----+-----+-----+-----+
| Kewal | 2 | 2750.00 | 1375.000000 |
+-----+-----+-----+-----+

```

```

sql> /*
-- =====
-- INNER, LEFT, RIGHT, FULL, SELF, CROSS JOIN Examples
-- =====
*/
OK, 0 records retrieved in 0.769ms

sql> use sql_zero_to_hero;
OK, 0 records retrieved in 0.888ms

sql> -- * Create Departments Table
-- CREATE TABLE departments_09 (
--   dept_id INT PRIMARY KEY,
--   dept_name VARCHAR(50)
-- );

OK, 0 records retrieved in 97.127ms

sql> -- * Create Employees Table
-- CREATE TABLE employees_09 (
--   emp_id INT PRIMARY KEY,
--   emp_name VARCHAR(50),
--   dept_id INT
-- );

OK, 0 records retrieved in 116.232ms

sql> -- * Insert data into departments
-- INSERT INTO departments_09 (dept_id, dept_name) VALUES
-- (1, 'IT'),
-- (2, 'HR'),
-- (3, 'Finance'),
-- (4, 'Marketing');
OK, 4 rows affected in 27.52ms

sql> -- * Insert data into employees
-- INSERT INTO employees_09 (emp_id, emp_name, dept_id) VALUES
-- (101, 'Suhas', 1),
-- (102, 'Rutuja', 1),
-- (103, 'Kewal', 2),
-- (104, 'Chirag', NULL),
-- (105, 'Sonal', 5); -- Invalid dept_id

OK, 5 rows affected in 19.452ms
OK, 0 records retrieved in 0.888ms

sql> -- =====
-- * INNER JOIN
-- Shows only matching rows from both tables
-- SELECT
--   emp.emp_name, dept.dept_name
-- FROM employees_09 emp
-- INNER JOIN departments_09 dept ON emp.dept_id = dept.dept_id;

emp_name | dept_name
Suhas    | IT
Rutuja   | IT
Kewal    | HR
OK, 3 records retrieved in 2.871ms

sql> -- =====
-- * LEFT JOIN
-- Shows all rows from left table + matching from right
-- SELECT
--   emp.emp_name, dept.dept_name
-- FROM employees_09 emp
-- LEFT JOIN departments_09 dept ON emp.dept_id = dept.dept_id;

emp_name | dept_name
Suhas    | IT
Rutuja   | IT
Kewal    | HR
Chirag   | NULL
Sonal    | NULL
OK, 5 records retrieved in 1.618ms

sql> -- =====
-- * RIGHT JOIN
-- Shows all rows from right table + matching from left
-- SELECT
--   emp.emp_name, dept.dept_name
-- FROM employees_09 emp
-- RIGHT JOIN departments_09 dept ON emp.dept_id = dept.dept_id;

emp_name | dept_name
Rutuja   | IT
Suhas    | IT
Kewal    | HR
NULL     | Finance
NULL     | Marketing
OK, 5 records retrieved in 1.332ms

sql> -- =====
-- * FULL OUTER JOIN
-- Combine LEFT + RIGHT using UNION
-- SELECT emp.emp_name, dept.dept_name
-- FROM employees_09 emp
-- LEFT JOIN departments_09 dept ON emp.dept_id = dept.dept_id
-- UNION
-- SELECT emp.emp_name, dept.dept_name
-- FROM employees_09 emp
-- RIGHT JOIN departments_09 dept ON emp.dept_id = dept.dept_id;

emp_name | dept_name
Suhas    | IT
Rutuja   | IT
Kewal    | HR
Chirag   | NULL
Sonal    | NULL
NULL     | Finance
NULL     | Marketing
OK, 7 records retrieved in 2.307ms

sql> -- =====
-- * SELF JOIN
-- Join the same table with itself
-- SELECT
--   e1.emp_name AS emp1,
--   e2.emp_name AS emp2
-- FROM employees_09 e1
-- JOIN employees_09 e2 ON e1.dept_id = e2.dept_id
-- WHERE e1.emp_id <> e2.emp_id;

emp1 | emp2
Rutuja | Suhas
Suhas | Rutuja
OK, 2 records retrieved in 1.417ms

sql> -- * CROSS JOIN
-- All possible combinations (Cartesian Product)
-- SELECT emp.emp_name, dept.dept_name
-- FROM employees_09 emp
-- CROSS JOIN departments_09 dept;

emp_name | dept_name
Suhas    | Marketing
Suhas    | Finance
Suhas    | HR
Suhas    | IT
Rutuja   | Marketing
Rutuja   | Finance
Rutuja   | HR
Rutuja   | IT
Kewal    | Marketing
Kewal    | Finance
Kewal    | HR
Kewal    | IT
Sonal    | Marketing
Sonal    | Finance
Sonal    | HR
Sonal    | IT

```

emp_name	dept_name
Kewal	Finance
Kewal	HR
Kewal	IT
Chirag	Marketing
Chirag	Finance
Chirag	HR
Chirag	IT
Sonal	Marketing
Sonal	Finance
Sonal	HR
Sonal	IT

OK, 20 records retrieved in 1.049ms

View:  

Pages:    

Activate Windows 



```

-- =====
-- Single-row, Multi-row, Correlated, EXISTS, NOT EXISTS
-- =====

*/
OK, 0 records retrieved in 0.012ms

sql> --> Use sql_zero_to_hero;
OK, 0 records retrieved in 0.762ms

sql> --> Create departments_10 table
--> CREATE TABLE departments_10 (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

OK, 0 records retrieved in 0.1686ms

sql> --> Create employees_10 table
--> CREATE TABLE employees_10 (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    salary INT
);

OK, 0 records retrieved in 0.7524ms

sql> --> Insert into departments_10
--> INSERT INTO departments_10 (dept_id, dept_name) VALUES
    (1, 'IT'),
    (2, 'HR'),
    (3, 'Finance'),
    (4, 'Marketing');

OK, 4 rows affected in 24.424ms

sql> --> Insert into employees_10
--> INSERT INTO employees_10 (emp_id, emp_name, dept_id, salary) VALUES
    (101, 'Suhas', 1, 50000),
    (102, 'Rutuja', 1, 55000),
    (103, 'Kawal', 2, 60000),
    (104, 'Chirag', 3, 65000),
    (105, 'Sonal', 4, 40000),
    (106, 'Asha', 2, 50000);

OK, 6 rows affected in 26.524ms

sql> --> View data
--> SELECT * FROM employees_10;

emp_id  emp_name  dept_id  salary
-----
101     Suhas     1        50000
102     Rutuja    1        55000
103     Kawal     2        60000
104     Chirag    3        65000
105     Sonal     4        40000
106     Asha      2        50000

OK, 6 records retrieved in 1.888ms

sql> --> SELECT * FROM departments_10;

dept_id  dept_name
-----
1        IT
2        HR
3        Finance
4        Marketing

OK, 4 records retrieved in 1.088ms

sql> --> SINGLE-ROW SUBQUERY
--> Find employees with salary equal to max salary

--> SELECT emp_name, salary
    FROM employees_10
   WHERE salary = (SELECT MAX(salary) FROM employees_10);

emp_name  salary
-----
Chirag     65000

OK, 1 record retrieved in 3.157ms

sql> --> MULTI-ROW SUBQUERY
--> Get names of employees who work in departments: HR or IT

--> SELECT emp_name
    FROM employees_10
   WHERE dept_id IN (SELECT dept_id FROM departments_10 WHERE dept_name IN ('IT', 'HR'));

emp_name
-----
Suhas
Rutuja
Kawal
Asha

OK, 4 records retrieved in 1.44ms

sql> --> CORRELATED SUBQUERY
--> Find employees whose salary is greater than average salary of their department

--> SELECT emp_name, dept_id, salary
    FROM employees_10 e
   WHERE salary > (
       SELECT AVG(salary)
         FROM employees_10
        WHERE dept_id = e.dept_id
    );

emp_name  dept_id  salary
-----
Rutuja     1        55000
Kawal      2        60000

OK, 2 records retrieved in 1.138ms

sql> --> EXISTS
--> Get departments that have at least one employee

--> SELECT dept_name
    FROM departments_10 d
   WHERE EXISTS (
       SELECT 1
         FROM employees_10 e
        WHERE e.dept_id = d.dept_id
    );

dept_name
-----
IT
HR
Finance
Marketing

OK, 4 records retrieved in 3.799ms

sql> --> NOT EXISTS
--> Get departments with no employees

--> SELECT dept_name
    FROM departments_10 d
   WHERE NOT EXISTS (
       SELECT 1
         FROM employees_10 e
        WHERE e.dept_id = d.dept_id
    );

```

```
sql> -- =====
CASE STATEMENT
-- CASE, WHEN, THEN, ELSE, END, Nested CASE

-- Searched CASE (most used)
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE resultn
END

-- Simple CASE (exact match)
CASE column
  WHEN value1 THEN result1
  WHEN value2 THEN result2
  ELSE resultn
END

--
--
OK, 0 records retrieved in 0.875ms

sql> -- use sql_zero_to_hero;
OK, 0 records retrieved in 1.032ms

sql> -- ♦ Create table
-- CREATE TABLE students_11 (
--   student_id INT PRIMARY KEY,
--   name VARCHAR(50),
--   marks INT,
--   city VARCHAR(50)
-- );

OK, 0 records retrieved in 93.722ms

sql> -- ♦ Insert data
-- INSERT INTO students_11 (student_id, name, marks, city)
-- VALUES
-- (1, 'Suhas', 85, 'Pune'),
-- (2, 'Rutuja', 72, 'Nagpur'),
-- (3, 'Kewal', 55, 'Mumbai'),
-- (4, 'Chirag', 48, 'Nanded'),
-- (5, 'Sonal', 90, 'Aurangabad');

OK, 5 rows affected in 23.907ms

sql> -- ♦ View data
-- SELECT * FROM students_11;

student_id | name      | marks | city
-----
1          | Suhas     | 85    | Pune
2          | Rutuja    | 72    | Nagpur
3          | Kewal     | 55    | Mumbai
4          | Chirag    | 48    | Nanded
5          | Sonal     | 90    | Aurangabad

OK, 5 records retrieved in 1.063ms

sql> -- ♦ CASE Statement: Simple Grading
-- Categorize students based on marks

-- SELECT
--   name,
--   marks,
--   CASE
--     WHEN marks >= 85 THEN 'A'
--     WHEN marks >= 70 THEN 'B'
--     WHEN marks >= 50 THEN 'C'
--     ELSE 'D'
--   END AS grade
-- FROM students_11;

name | marks | grade
-----
Suhas | 85    | A
Rutuja | 72    | B
Kewal | 55    | C
Chirag | 48    | D
Sonal | 90    | A

OK, 5 records retrieved in 1.475ms

sql> -- ♦ CASE WITH ELSE
-- Mark city-based remarks (basic search CASE)

-- SELECT
--   name,
--   city,
--   CASE city
--     WHEN 'Pune' THEN 'HQ City'
--     WHEN 'Mumbai' THEN 'Metro'
--     ELSE 'Other City'
--   END AS city_type
-- FROM students_11;

name | city      | city_type
-----
Suhas | Pune      | HQ City
Rutuja | Nagpur    | Other City
Kewal | Mumbai    | Metro
Chirag | Nanded    | Other City
Sonal | Aurangabad | Other City

OK, 5 records retrieved in 1.008ms

sql> -- =====
-- ♦ Nested CASE
-- Grade + Remark based on nested conditions

-- SELECT
--   name,
--   marks,
--   CASE
--     WHEN marks >= 85 THEN
--       CASE
--         WHEN marks = 90 THEN 'Perfect A+'
--         ELSE 'Strong A'
--       END
--     WHEN marks >= 70 THEN 'Good B'
--     WHEN marks >= 50 THEN 'Average C'
--     ELSE 'Poor D'
--   END AS remarks
-- FROM students_11;

name | marks | remarks
-----
Suhas | 85    | Strong A
Rutuja | 72    | Good B
Kewal | 55    | Average C
Chirag | 48    | Poor D
Sonal | 90    | Perfect A+
```

```

sql> -- TIME FUNCTIONS
-- CURDATE(), NOW(), DATE(), DATEDIFF(), STR_TO_DATE(), DATE_ADD(), DATE_SUB()
/*

| Function | Description |
|-----|-----|
| CURDATE() | Returns current date (YYYY-MM-DD) |
| NOW() | Returns current datetime (YYYY-MM-DD hh:mm:ss) |
| DATE() | Extracts date from datetime |
| DATEDIFF(a, b) | Days between a and b |
| STR_TO_DATE() | Converts string to date format |
| DATE_ADD() | Adds interval to date |
| DATE_SUB() | Subtracts interval from date |
*/

OK, 0 records retrieved in 1.12ms

sql> -- USE sql_zero_to_hero;
OK, 0 records retrieved in 0.77ms

sql> -- ♦ Create table with dates
-- CREATE TABLE events_12 (
--     event_id INT PRIMARY KEY,
--     event_name VARCHAR(50),
--     event_date DATE
-- );

OK, 0 records retrieved in 79.92ms

sql> -- ♦ Insert sample dates
-- INSERT INTO events_12 (event_id, event_name, event_date)
-- VALUES
-- (1, 'SQL Workshop', '2024-12-01'),
-- (2, 'Python Bootcamp', '2024-12-10'),
-- (3, 'Hackathon', '2024-11-25'),
-- (4, 'AI Seminar', '2024-12-20'),
-- (5, 'Webinar', '2024-11-30');

OK, 5 rows affected in 25.725ms

sql> -- ♦ View table
-- SELECT * FROM events_12;

+----+-----+-----+
| event_id | event_name | event_date |
+----+-----+-----+
| 1 | SQL Workshop | 12/01/2024 |
| 2 | Python Bootcamp | 12/10/2024 |
| 3 | Hackathon | 11/25/2024 |
| 4 | AI Seminar | 12/20/2024 |
| 5 | Webinar | 11/30/2024 |
+----+-----+-----+

OK, 5 records retrieved in 1.059ms

sql> -- ♦ CURDATE()
-- Returns current date only
-- SELECT CURDATE() AS today_date;

+-----+
| today_date |
+-----+
| 07/09/2025 |
+-----+

OK, 1 record retrieved in 2.988ms

sql> -- ♦ NOW()
-- Returns current date and time
-- SELECT NOW() AS current_datetime;

+-----+
| current_datetime |
+-----+
| 07/09/2025 |
+-----+

OK, 1 record retrieved in 1.438ms

sql> -- ♦ DATE()
-- Extract only the date part from DATETIME
-- SELECT DATE(NOW()) AS only_date;

+-----+
| only_date |
+-----+
| 07/09/2025 |
+-----+

OK, 1 record retrieved in 1.581ms

sql> -- ♦ DATE()
-- Extract only the date part from DATETIME
-- SELECT DATE(NOW()) AS only_date;

+-----+
| only_date |
+-----+
| 07/09/2025 |
+-----+

OK, 1 record retrieved in 0.819ms

sql> -- ♦ STR_TO_DATE()
-- Convert string to date format
-- SELECT STR_TO_DATE('08-07-2025', '%d-%m-%Y') AS converted_date;

+-----+
| converted_date |
+-----+
| 07/08/2025 |
+-----+

OK, 1 record retrieved in 1.021ms

sql> -- ♦ DATE_ADD()
-- Add 10 days to current date
-- SELECT
--     CURDATE() AS today,
--     DATE_ADD(CURDATE(), INTERVAL 10 DAY) AS plus_10_days;

+-----+ +-----+
| today | plus_10_days |
+-----+ +-----+
| 07/09/2025 | 07/19/2025 |
+-----+ +-----+

OK, 1 record retrieved in 1.189ms

sql> -- ♦ DATE_SUB()
-- Subtract 5 days from current date
-- SELECT
--     CURDATE() AS today,
--     DATE_SUB(CURDATE(), INTERVAL 5 DAY) AS minus_5_days;

+-----+ +-----+
| today | minus_5_days |
+-----+ +-----+
| 07/09/2025 | 07/04/2025 |
+-----+ +-----+

OK, 1 record retrieved in 1.014ms

sql> -- ♦ Real-life use: upcoming Events only
-- SELECT
--     event_name,
--     event_date,
--     DATEDIFF(event_date, CURDATE()) AS days_to_go
-- FROM events_12
-- WHERE event_date > CURDATE();

```

```
sql> */
```

◆ What is Indexing?

Indexes in MySQL are used to speed up the retrieval of rows by using a pointer. Without an index, MySQL must scan the entire table (called full table scan).

GENERAL SYNTAX

-- Create a regular index

```
CREATE INDEX index_name ON table_name (column_name);
```

-- Create a unique index (no duplicate values allowed)

```
CREATE UNIQUE INDEX index_name ON table_name (column_name);
```

-- Drop an index

```
DROP INDEX index_name ON table_name;
```

-- Show indexes of a table

```
SHOW INDEX FROM table_name;
```

```
*/
```

OK, 0 records retrieved in 1.963ms

```
sql> USE sql_zero_to_hero;
```

OK, 0 records retrieved in 0.755ms

```
sql> CREATE TABLE employees_13 (
```

```
    emp_id INT PRIMARY KEY,
```

```
    name VARCHAR(100),
```

```
    department VARCHAR(50),
```

```
    email VARCHAR(100),
```

```
    salary INT
```

```
);
```

OK, 0 records retrieved in 87.846ms

```
sql> INSERT INTO employees_13 (emp_id, name, department, email, salary)
```

```
VALUES
```

```
(1, 'Suhas', 'IT', 'suhas@xyz.com', 70000),
```

```
(2, 'Rutuja', 'HR', 'rutuja@xyz.com', 60000),
```

```
(3, 'Kewal', 'IT', 'kewal@xyz.com', 75000),
```

```
(4, 'Chirag', 'Finance', 'chirag@xyz.com', 80000),
```

```
(5, 'Sonal', 'HR', 'sonal@xyz.com', 50000);
```

OK, 5 rows affected in 26.594ms

```
sql> -- 🐞 Step 2: Select (Unindexed)
```

```
• EXPLAIN SELECT * FROM employees_13 WHERE department = 'IT';
```

id	select_type	table	partitions	type	possible_keys	key	key_len
1	SIMPLE	employees_13	NULL	ALL	NULL	NULL	NULL

OK, 1 record retrieved in 1.884ms

```
sql> CREATE INDEX idx_department ON employees_13(department);
```

OK, 0 records retrieved in 116.253ms

```
sql> -- 🐞 Step 4: Select (With Index)
```

```
• EXPLAIN SELECT * FROM employees_13 WHERE department = 'IT';
```

```
• -- ✅ type: ref → Index used
```

```
-- ✅ key: idx_department → Index applied
```

id	select_type	table	partitions	type	possible_keys	key	key_len
1	SIMPLE	employees_13	NULL	ref	idx_department	idx_department	203

OK, 1 record retrieved in 1.399ms

```
sql> -- 🐞 Step 5: Unique Index
```

```
• CREATE UNIQUE INDEX idx_email ON employees_13(email);
```

OK, 0 records retrieved in 170.067ms

```
sql> -- ❌ Step 6: Duplicate Entry (Test Unique Index)
```

```
• INSERT INTO employees_13 VALUES (6, 'Ravi', 'IT', 'suhas@xyz.com', 60000);
```

MySQL Error (1062): Duplicate entry 'suhas@xyz.com' for key 'employees_13.idx_email'

```
sql> -- 🐞 Step 8: Show Indexes
```

```
• SHOW INDEX FROM employees_13;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part
employees_13	0	PRIMARY	1	emp_id	A	5	NULL
employees_13	0	idx_email	1	email	A	5	NULL
employees_13	1	idx_department	1	department	A	3	NULL

OK, 3 records retrieved in 44.181ms

```
sql> -- 🗑️ Step 7: Drop Index
```

```
• DROP INDEX idx_department ON employees_13;
```

String Function

Function	Description
LENGTH()	Byte length of string
CHAR_LENGTH()	Character count
CONCAT()	Combine strings
UPPER()	Convert to uppercase
LOWER()	Convert to lowercase
SUBSTRING()	Extract substring
REPLACE()	Replace part of string
TRIM()	Remove leading/trailing whitespace

```

--
OK, 0 records retrieved in 1.888ms
sql> USE sql_zero_to_hero;
OK, 0 records retrieved in 1.123ms
sql> CREATE TABLE customers_14 (
    cust_id INT PRIMARY KEY,
    full_name VARCHAR(100),
    email VARCHAR(100),
    city VARCHAR(50)
);
OK, 0 records retrieved in 123.185ms
sql> INSERT INTO customers_14 (cust_id, full_name, email, city)
VALUES
(1, 'suhas shende', 'SUHAS@EMAIL.COM', 'Pune'),
(2, 'Rutuja Wagh', 'rutuja@email.com', 'Nagpur'),
(3, 'kewal patil', 'kewal@email.com', 'Mumbai'),
(4, 'chirag Jain', 'chirag@email.com', 'Nanded'),
(5, 'Sonal Mehta', 'sonal@email.com', 'Aurangabad');
OK, 5 rows affected in 52.767ms

```

```

sql> -- • LENGTH() → Bytes count
• SELECT full_name, LENGTH(full_name) AS byte_length FROM customers_14;

```

full_name	byte_length
suhas shende	16
Rutuja Wagh	11
kewal patil	12
chirag Jain	11
Sonal Mehta	15

```

sql> -- • CHAR_LENGTH() → Character count
• SELECT full_name, CHAR_LENGTH(full_name) AS char_len FROM customers_14;

```

full_name	char_len
suhas shende	16
Rutuja Wagh	11
kewal patil	12
chirag Jain	11
Sonal Mehta	15

```

sql> -- • CONCAT() → Join strings
• SELECT CONCAT(full_name, ' - ', city) AS full_detail FROM customers_14;

```

full_detail
suhas shende - Pune
Rutuja Wagh - Nagpur
kewal patil - Mumbai
chirag Jain - Nanded
Sonal Mehta - Aurangabad

```

sql> -- • UPPER() → Uppercase
• SELECT full_name, UPPER(full_name) AS upper_name FROM customers_14;

```

full_name	upper_name
suhas shende	SUHAS SHENDE
Rutuja Wagh	RUTUJA WAGH
kewal patil	KEWAL PATIL
chirag Jain	CHIRAG JAIN
Sonal Mehta	SONAL MEHTA

```

sql> -- • LOWER() → Lowercase
• SELECT email, LOWER(email) AS lower_email FROM customers_14;

```

email	lower_email
SUHAS@EMAIL.COM	suhas@email.com
rutuja@email.com	rutuja@email.com
kewal@email.com	kewal@email.com
chirag@email.com	chirag@email.com
Sonal@email.com	sonal@email.com

```

sql> -- • SUBSTRING() → Part of string
• SELECT full_name, SUBSTRING(full_name, 1, 5) AS short_name FROM customers_14;

```

full_name	short_name
suhas shende	suh
Rutuja Wagh	Rutuj
kewal patil	kewal
chirag Jain	chira
Sonal Mehta	Sonal

```

sql> -- • REPLACE() → Replace text
• SELECT email, REPLACE(email, 'EMAIL', '@gmail') AS fixed_email FROM customers_14;

```

email	fixed_email
SUHAS@EMAIL.COM	SUHAS@gmail.COM
rutuja@email.com	rutuja@gmail.com
kewal@email.com	kewal@gmail.com
chirag@email.com	chirag@gmail.com
Sonal@email.com	Sonal@gmail.com

```

sql> -- • TRIM() → Remove spaces from both ends
• SELECT full_name, TRIM(full_name) AS trimmed_name FROM customers_14;

```

full_name	trimmed_name
suhas shende	suhas shende
Rutuja Wagh	Rutuja Wagh
kewal patil	kewal patil
chirag Jain	chirag Jain
Sonal Mehta	Sonal Mehta

sql> */

Math Functions

Function	Use
ROUND()	Round number to decimal places
FLOOR()	Round down
CEIL()	Round up
ABS()	Absolute value
MOD()	Remainder of division
POWER()	Exponential (x^y)

*/

OK, 0 records retrieved in 1.483ms

sql> USE sql_zero_to_hero;

OK, 0 records retrieved in 1.778ms

sql> CREATE TABLE salaries_15 (

```
    emp_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    base_salary FLOAT,  
    bonus FLOAT  
);
```

OK, 0 records retrieved in 112.425ms

sql> INSERT INTO salaries_15 (emp_id, name, base_salary, bonus)
VALUES

```
(1, 'Suhas', 55432.75, 1200.55),  
(2, 'Rutuja', 68450.80, 2345.40),  
(3, 'Kewal', -45000.25, 1100.00),  
(4, 'Chirag', 32999.99, 500.99),  
(5, 'Sonal', 74320.90, 3300.60);
```

OK, 5 rows affected in 38.637ms

sql> -- ♦ ROUND(number, decimal_places)

♦ SELECT name, base_salary, ROUND(base_salary, 1) AS rounded_salary FROM salaries_15;

name	base_salary	rounded_salary
Suhas	55432.8	55432.8
Rutuja	68450.8	68450.8
Kewal	-45000.2	-45000.2
Chirag	33000	33000
Sonal	74320.9	74320.9

OK, 5 records retrieved in 25.925ms

sql> -- ♦ FLOOR(number) → Round down

♦ SELECT name, FLOOR(base_salary) AS floor_salary FROM salaries_15;

name	floor_salary
Suhas	55432
Rutuja	68450
Kewal	-45001
Chirag	32999
Sonal	74320

OK, 5 records retrieved in 1.005ms

sql> -- ♦ CEIL(number) → Round up

♦ SELECT name, CEIL(base_salary) AS ceil_salary FROM salaries_15;

name	ceil_salary
Suhas	55433
Rutuja	68451
Kewal	-45000
Chirag	33000
Sonal	74321

OK, 5 records retrieved in 1.48ms

sql> -- ♦ ABS(number) → Absolute value

♦ SELECT name, base_salary, ABS(base_salary) AS abs_salary FROM salaries_15;

name	base_salary	abs_salary
Suhas	55432.8	55432.75
Rutuja	68450.8	68450.796875
Kewal	-45000.2	45000.25
Chirag	33000	32999.98828125
Sonal	74320.9	74320.8984375

OK, 5 records retrieved in 1.656ms

sql> -- ♦ MOD(x, y) → Remainder of division

♦ SELECT name, MOD(emp_id, 2) AS is_even FROM salaries_15;

name	is_even
Suhas	1
Rutuja	0
Kewal	1
Chirag	0
Sonal	1

OK, 5 records retrieved in 1.577ms

sql> -- ♦ POWER(base, exponent)

♦ SELECT name, POWER(2, emp_id) AS power_value FROM salaries_15;

name	power_value
Suhas	2
Rutuja	4
Kewal	8
Chirag	16
Sonal	32

```
--*/
Window Functions
A function that performs calculations across a set of table rows that are related to the current row.
```

Function	Purpose
ROW_NUMBER()	Unique row index per group
RANK()	Rank with gaps for ties
DENSE_RANK()	Rank without gaps
NTILE(n)	Divides result set into n groups
LEAD()	Get next row's value
LAG()	Get previous row's value
OVER()	Mandatory for all window functions
PARTITION BY	Logical grouping for each function

```
OK, 0 records retrieved in 1.078ms
sql> +USE sql_care_to_hero;
```

```
OK, 0 records retrieved in 1.281ms
sql>
```

```
+CREATE TABLE employee_sales_16 (
  emp_id INT,
  emp_name VARCHAR(50),
  department VARCHAR(50),
  monthly_sales INT
);
```

```
OK, 0 records retrieved in 116.387ms
```

```
sql> +INSERT INTO employee_sales_16 (emp_id, emp_name, department, monthly_sales)
VALUES
(1, 'Suhas', 'IT', 50000),
(2, 'Rutuja', 'HR', 42000),
(3, 'Kewal', 'IT', 50000),
(4, 'Chirag', 'Finance', 55000),
(5, 'Sonal', 'HR', 47000),
(6, 'Tushar', 'Finance', 48000),
(7, 'Amruta', 'IT', 52000),
(8, 'Nikita', 'HR', 47000);
```

```
OK, 8 rows affected in 34.491ms
```

```
sql> -- + ROW_NUMBER() + Unique row numbering within partition
+SELECT emp_name, department, monthly_sales,
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY monthly_sales DESC) AS row_num
FROM employee_sales_16;
```

emp_name	department	monthly_sales	row_num
Chirag	Finance	55000	1
Tushar	Finance	48000	2
Sonal	HR	47000	1
Nikita	HR	47000	2
Rutuja	HR	42000	3
Amruta	IT	52000	1
Suhas	IT	50000	2
Kewal	IT	50000	3

```
OK, 8 records retrieved in 7.588ms
```

```
sql> -- + RANK() + Same rank for ties, but skips next
```

```
+SELECT emp_name, department, monthly_sales,
       RANK() OVER (PARTITION BY department ORDER BY monthly_sales DESC) AS rank_val
FROM employee_sales_16;
```

emp_name	department	monthly_sales	rank_val
Chirag	Finance	55000	1
Tushar	Finance	48000	2
Sonal	HR	47000	1
Nikita	HR	47000	1
Rutuja	HR	42000	3
Amruta	IT	52000	1
Suhas	IT	50000	2
Kewal	IT	50000	2

```
OK, 8 records retrieved in 1.839ms
```

```
sql> +SELECT emp_name, department, monthly_sales,
       DENSE_RANK() OVER (PARTITION BY department ORDER BY monthly_sales DESC) AS dense_rank_val
FROM employee_sales_16;
```

emp_name	department	monthly_sales	dense_rank_val
Chirag	Finance	55000	1
Tushar	Finance	48000	2
Sonal	HR	47000	1
Nikita	HR	47000	1
Rutuja	HR	42000	2
Amruta	IT	52000	1
Suhas	IT	50000	2
Kewal	IT	50000	2

```
OK, 8 records retrieved in 2.894ms
```

```
sql> -- + LEAD() + Next row value in same partition
```

```
+SELECT emp_name, department, monthly_sales,
       LEAD(monthly_sales) OVER (PARTITION BY department ORDER BY monthly_sales DESC) AS next_sales
FROM employee_sales_16;
```

emp_name	department	monthly_sales	next_sales
Chirag	Finance	55000	48000
Tushar	Finance	48000	NULL
Sonal	HR	47000	47000
Nikita	HR	47000	42000
Rutuja	HR	42000	NULL
Amruta	IT	52000	50000
Suhas	IT	50000	50000
Kewal	IT	50000	NULL

```
OK, 8 records retrieved in 8.587ms
```

```
sql> -- + LAG() + Previous row value
```

```
+SELECT emp_name, department, monthly_sales,
       LAG(monthly_sales) OVER (PARTITION BY department ORDER BY monthly_sales DESC) AS prev_sales
FROM employee_sales_16;
```

emp_name	department	monthly_sales	prev_sales
Chirag	Finance	55000	NULL
Tushar	Finance	48000	52000
Sonal	HR	47000	NULL
Nikita	HR	47000	47000
Rutuja	HR	42000	47000
Amruta	IT	52000	NULL
Suhas	IT	50000	52000
Kewal	IT	50000	50000

```
OK, 8 records retrieved in 1.492ms
```

```
sql> -- + NTILE(n) + Divide into buckets (quarters/percentiles)
```

```
+SELECT emp_name, monthly_sales,
       NTILE(4) OVER (ORDER BY monthly_sales DESC) AS sales_quartile
FROM employee_sales_16;
```

emp_name	monthly_sales	sales_quartile
Chirag	55000	1
Amruta	52000	1
Suhas	50000	2
Kewal	50000	2
Sonal	47000	3
Nikita	47000	3
Rutuja	42000	4
Tushar	48000	4

```
OK, 8 records retrieved in 4.643ms
```